

13-模型服务：怎样把你的离线模型部署到线上？

你好，我是王喆。今天我们来讨论“模型服务”（Model serving）。

在实验室的环境下，我们经常使用Spark MLlib、TensorFlow、PyTorch这些流行的机器学习库来训练模型，因为不用直接服务用户，所以往往得到一些离线的训练结果就觉得大功告成了。但在业界的生产环境中，模型需要在线上运行，实时地根据用户请求生成模型的预估值。这个把模型部署在线上环境，并实时进行模型推断（Inference）的过程就是模型服务。

模型服务对于推荐系统来说是至关重要的线上服务，缺少了它，离线的模型只能在离线环境里面“干着急”，不能发挥功能。但是，模型服务的方法可谓是五花八门，各个公司为了部署自己的模型也是各显神通。那么，业界主流的模型服务方法都有哪些，我们又该如何选择呢？

今天，我就带你学习主流的模型服务方法，并通过TensorFlow Serving把你的模型部署到线上。

业界的主流模型服务方法

由于各个公司技术栈的特殊性，采用不同的机器学习平台，模型服务的方法会截然不同，不仅如此，使用不同的模型结构和模型存储方式，也会让模型服务的方法产生区别。总的来说，那业界主流的模型服务方法有4种，分别是预存推荐结果或Embedding结果、预训练Embedding+轻量级线上模型、PMML模型以及TensorFlow Serving。接下来，我们就详细讲讲这些方法的实现原理，通过对比它们的优缺点，相信你会找到最合适自己业务场景的方法。

预存推荐结果或Embedding结果

对于推荐系统线上服务来说，最简单直接的模型服务方法就是在离线环境下生成对每个用户的推荐结果，然后将结果预存到以Redis为代表的线上数据库中。这样，我们在线上环境直接取出预存数据推荐给用户即可。

这个方法的优缺点都非常明显，我把它们总结在了下图中，你可以看看。



优点：

- 无须实现模型线上推断过程，线下训练平台与线上服务平台完全解耦，可以灵活地选择任意离线机器学习工具进行模型训练
- 线上服务过程没有复杂计算，推荐系统的线上延迟极低



缺点：

- 由于需要存储用户x 物品 x应用场景的组合推荐结果，在用户数量、物品数量等规模过大后，容易发生组合爆炸的情况，线上数据库根本无力支撑如此大规模结果的存储
- 无法引入线上场景（context）类特征，推荐系统的灵活性和效果受限



由于这些优缺点的存在，这种直接存储推荐结果的方式往往只适用于用户规模较小，或者一些冷启动、热门榜单等特殊的应用场景中。

那如果在用户规模比较大的场景下，我们该怎么减少模型存储所需的空間呢？我们其实可以通过存储Embedding的方式来替代直接存储推荐结果。具体来说就是，我们先离线训练好Embedding，然后在线上

通过相似度运算得到最终的推荐结果。

在前面的课程中，我们通过Item2vec，Graph Embedding等方法生成物品Embedding，再存入Redis供线上使用的过程，这就是预存Embedding的模型服务方法的典型应用。

由于，线上推断过程非常简单快速，因此，预存Embedding的方法是业界经常采用的模型服务手段。但它的局限性同样存在，由于完全基于线下计算出Embedding，这样的方式无法支持线上场景特征的引入，并且无法进行复杂模型结构的线上推断，表达能力受限。因此对于复杂模型，我们还需要从模型实时线上推断的角度入手，来改进模型服务的方法。

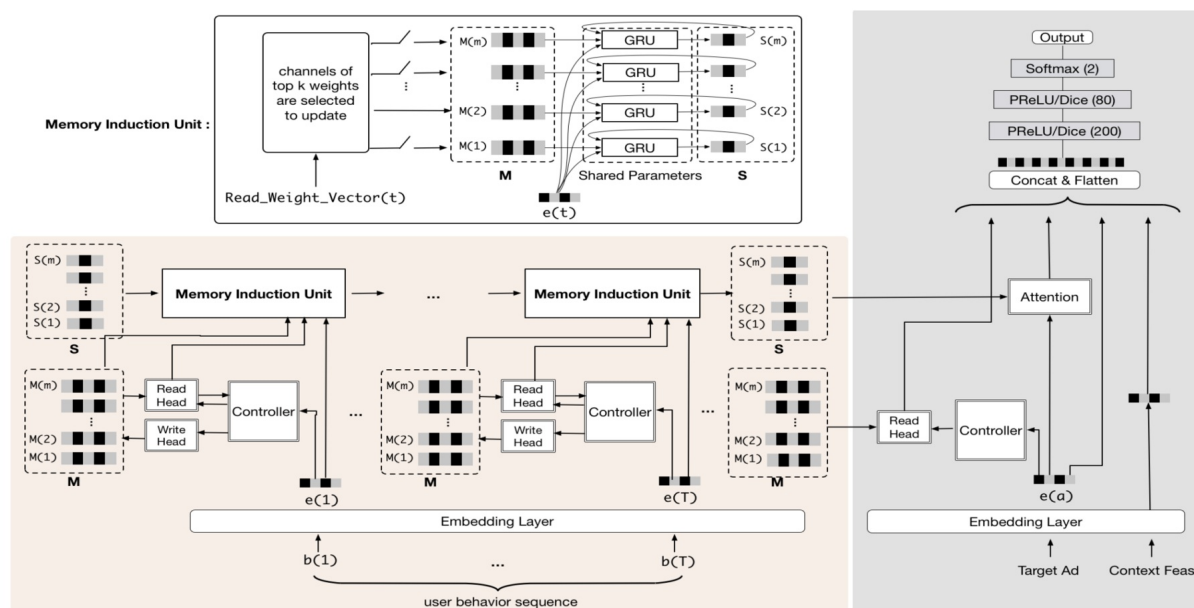
预训练Embedding+轻量级线上模型

事实上，直接预存Embedding的方法让模型表达能力受限这个问题的产生，主要是因为我们仅仅采用了“相似度计算”这样非常简单的方式去得到最终的推荐分数。既然如此，那我们能不能在线上实现一个比较复杂的操作，甚至是用神经网络来生成最终的预估值呢？当然是可行的，这就是业界很多公司采用的“预训练Embedding+轻量级线上模型”的模型服务方式。

详细一点来说，这样的服务方式指的是“**用复杂深度学习网络离线训练生成Embedding，存入内存数据库，再在线上实现逻辑回归或浅层神经网络等轻量级模型来拟合优化目标**”。

口说无凭，接下来，我们就来看一个业界实际的例子。我们先来看看下面这张模型结构图，这是阿里的推荐模型MIMN（Multi-channel user Interest Memory Network，多通道用户兴趣记忆网络）的结构。神经网络，才是真正在线上服务的部分。

仔细看这张图你会注意到，左边粉色的部分是复杂模型部分，右边灰色的部分是简单模型部分。看这张图的时候，其实你不需要纠结于复杂模型的结构细节，你只要知道左边的部分不管多复杂，它们其实是在线下训练生成的，而右边的部分是一个经典的多层神经网络，它才是真正在线上服务的部分。



这两部分的接口在哪里呢？你可以看一看图中连接处的位置，有两个被虚线框框住的数据结构，分别是 $S(1)-S(m)$ 和 $M(1)-M(m)$ 。它们其实就是在离线生成的Embedding向量，在MIMN模型中，它们被称为“多通道用户兴趣向量”，这些Embedding向量就是连接离线模型和线上模型部分的接口。

线上部分从Redis之类的模型数据库中拿到这些离线生成Embedding向量，然后跟其他特征的Embedding向量组合在一起，扔给一个标准的多层神经网络进行预估，这就是一个典型的“预训练Embedding+轻量级线上模型”的服务方式。

它的好处显而易见，就是我们隔离了离线模型的复杂性和线上推断的效率要求，离线环境下，你可以尽情地使用复杂结构构建你的模型，只要最终的结果是Embedding，就可以轻松地供给线上推断使用。

利用PMML转换和部署模型

虽然Embedding+轻量级模型的方法既实用又高效，但它还是把模型进行了割裂，让模型不完全是End2End（端到端）训练+End2End部署这种最“完美”的方式。那有没有能够在离线训练完模型之后什么都不用做，直接部署模型的方式呢？当然是有的，也就是我接下来要讲的脱离于平台的通用模型部署方式，PMML。

PMML的全称是“预测模型标记语言” (Predictive Model Markup Language, PMML)，它是一种通用的以XML的形式表示不同模型结构参数的标记语言。在模型上线的过程中，PMML经常作为中间媒介连接离线训练平台和线上预测平台。

这么说可能还比较抽象。接下来，我就以Spark MLlib模型的训练和上线过程为例，来和你详细解释一下，PMML在整个机器学习模型训练及上线流程中扮演的角色。

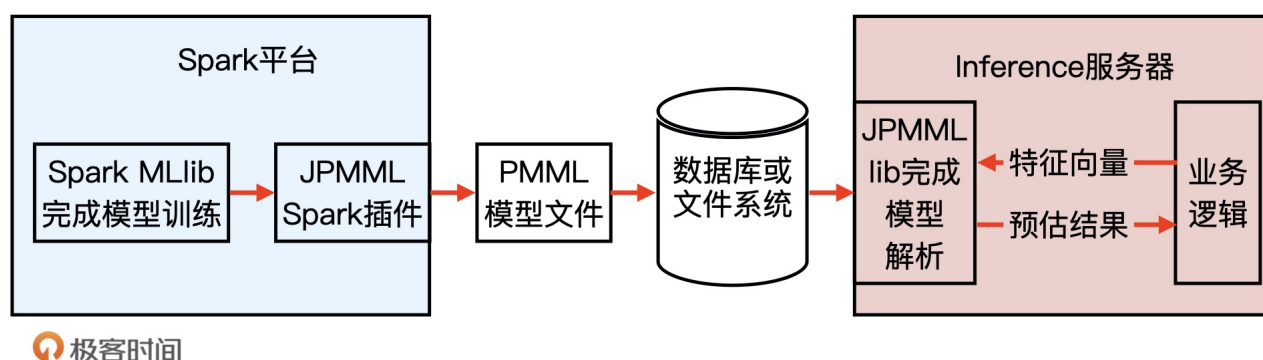


图3中的例子使用了JPMML作为序列化和解析PMML文件的library（库），JPMML项目分为Spark和Java Server两部分。Spark部分的library完成Spark MLlib模型的序列化，生成PMML文件，并且把它保存到线上服务器能够触达的数据库或文件系统中，而Java Server部分则完成PMML模型的解析，生成预估模型，完成了与业务逻辑的整合。

JPMML在Java Server部分只进行推断，不考虑模型训练、分布式部署等一系列问题，因此library比较轻，能够高效地完成推断过程。与JPMML相似的开源项目还有MLeap，同样采用了PMML作为模型转换和上线的媒介。

事实上，JPMML和MLeap也具备Scikit-learn、TensorFlow等简单模型的转换和上线能力。我把[JPMML](#)和[MLeap](#)的项目地址放在这里，感兴趣的同学可以进一步学习和实践。

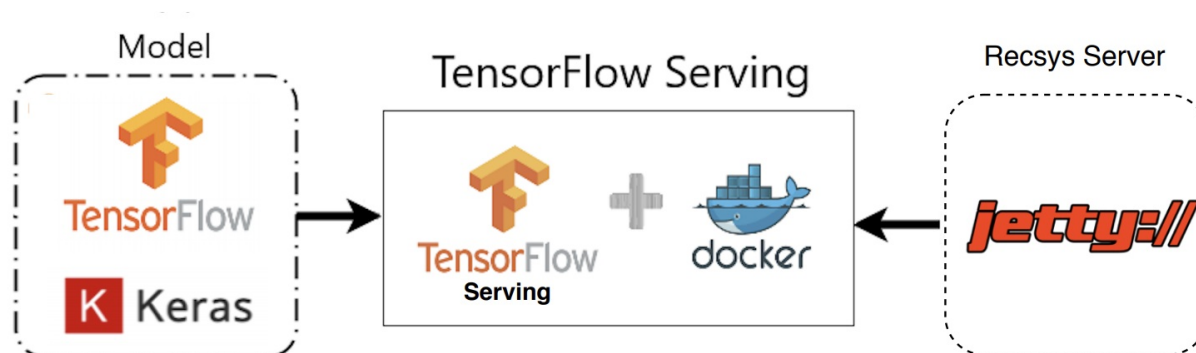
TensorFlow Serving

既然PMML已经是End2End训练+End2End部署这种最“完美”的方式了，那我们的课程中为什么不使用它进行模型服务呢？这是因为对于具有复杂结构的深度学习模型来说，PMML语言的表示能力还是比较有限的，还不足以支持复杂的深度学习模型结构。由于咱们课程中的推荐模型篇，会主要使用TensorFlow来构

建深度学习推荐模型，这个时候PMML的能力就有点不足了。想要上线TensorFlow模型，我们就需要借助TensorFlow的原生模型服务模块，也就是TensorFlow Serving的支持。

从整体工作流程来看，TensorFlow Serving和PMML类工具的流程一致，它们都经历了模型存储、模型载入还原以及提供服务的过程。在具体细节上，TensorFlow在离线把模型序列化，存储到文件系统，TensorFlow Serving把模型文件载入到模型服务器，还原模型推断过程，对外以http接口或gRPC接口的方式提供模型服务。

再具体到咱们的SparrowRecsys项目中，我们会在离线使用TensorFlow的Keras接口完成模型构建和训练，再利用TensorFlow Serving载入模型，用docker作为服务容器，然后在Jetty推荐服务器中发出http请求到TensorFlow Serving，获得模型推断结果，最后推荐服务器利用这一结果完成推荐排序。



实战搭建TensorFlow Serving模型服务

好了，清楚了模型服务的相关知识，相信你对各种模型服务方法的优缺点都已经了然于胸了。刚才我们提到，咱们的课程选用了TensorFlow作为构建深度学习推荐模型的主要平台，并且选用了TensorFlow Serving作为模型服务的技术方案，它们可以说是整个推荐系统的核心了。那为了给之后的学习打下基础，接下来，我就带你搭建一个TensorFlow Serving的服务，把这部分重点内容牢牢掌握住。

总的来说，搭建一个TensorFlow Serving的服务主要有3步，分别是安装Docker，建立TensorFlow Serving服务，以及请求TensorFlow Serving获得预估结果。为了提高咱们的效率，我希望你能打开电脑跟着我的讲解和文稿里的指令代码，一块儿来安装。

1. 安装Docker

TensorFlow Serving最普遍、最便捷的服务方式就是使用Docker建立模型服务API。为了方便你后面的学习，我再简单说说Docker。Docker是一个开源的应用容器引擎，你可以把它当作一个轻量级的虚拟机。它可以让开发者打包他们的应用以及依赖包到一个轻量级、可移植的容器中，然后发布到任何流行的操作系统，比如Linux/Windows/Mac的机器上。Docker容器相互之间不会有任何接口，而且容器本身的开销极低，这就让Docker成为了非常灵活、安全、伸缩性极强的计算资源平台。

因为TensorFlow Serving对外提供的是模型服务接口，所以使用Docker作为容器的好处主要有两点，一是可以非常方便的安装，二是在模型服务的压力变化时，可以灵活地增加或减少Docker容器的数量，做到弹性计算，弹性资源分配。Docker的安装也非常简单，我们参考[官网的教程](#)，像安装一个普通软件一样下载安装就好。

安装完Docker后，你不仅可以通过图形界面打开并运行Docker，而且可以通过命令行来进行docker相关的

操作。那怎么验证你是否安装成功了呢？只要你打开命令行输入docker --version命令，它能显示出类似“Docker version 19.03.13, build 4484c46d9d”这样的版本号，就说明你的Docker环境已经准备好了。

2. 建立TensorFlow Serving服务

Docker环境准备好之后，我们就可以着手建立TensorFlow Serving服务了。

首先，我们要利用Docker命令拉取TensorFlow Serving的镜像：

```
# 从docker仓库中下载tensorflow/serving镜像
docker pull tensorflow/serving
```

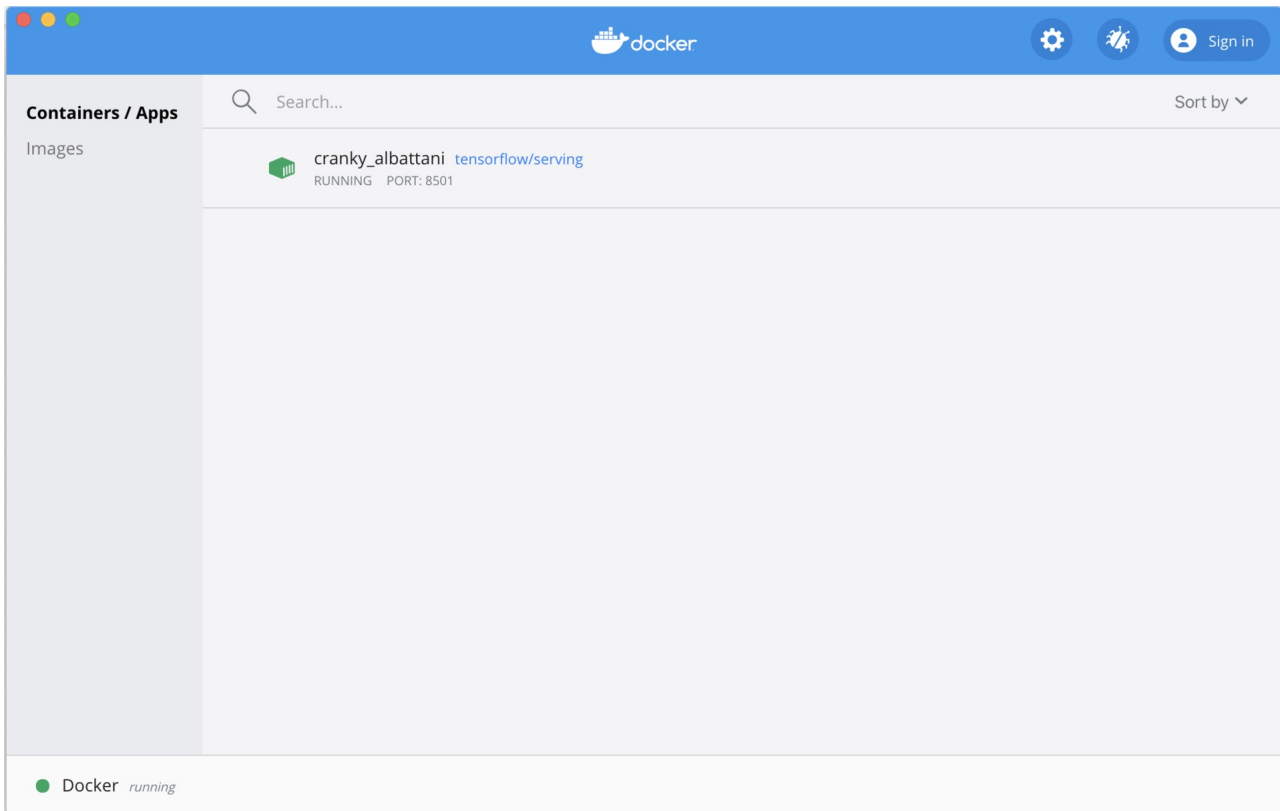
然后，我们再从Tensorflow的官方GitHub地址下载TensorFlow Serving相关的测试模型文件：

```
# 把tensorflow/serving的测试代码clone到本地
git clone https://github.com/tensorflow/serving
# 指定测试数据的地址
TESTDATA="$(pwd)/serving/tensorflow_serving/servables/tensorflow/testdata"
```

最后，我们在Docker中启动一个包含TensorFlow Serving的模型服务容器，并载入我们刚才下载的测试模型文件half_plus_two：

```
# 启动TensorFlow Serving容器，在8501端口运行模型服务api
docker run -t --rm -p 8501:8501 \
  -v "$TESTDATA/saved_model_half_plus_two_cpu:/models/half_plus_two" \
  -e MODEL_NAME=half_plus_two \
  tensorflow/serving &
```

在命令执行完成后，如果你在Docker的管理界面中看到了Tensorflow Serving容器，如下图所示，就证明TensorFlow Serving服务被你成功建立起来了。



3. 请求TensorFlow Serving获得预估结果

最，我们再来验证一下是否能够通过http请求从TensorFlow Serving api中获得模型的预估结果。我们可以通过curl命令来发送http post请求到TensorFlow Serving的地址，或者利用postman等软件来组装post请求进行验证。

```
# 请求模型服务api
curl -d '{"instances": [1.0, 2.0, 5.0]}' \
  -X POST http://localhost:8501/v1/models/half_plus_two:predict
```

如果你看到了下图这样的返回结果，就说明TensorFlow Serving服务已经成功建立起来了。

```
# 返回模型推断结果如下
# Returns => { "predictions": [2.5, 3.0, 4.5] }
```

如果对这个整个过程还有疑问的话，你也可以参考Tensorflow serving的[官方教程](#)。

不过，有一点我还想提醒你，这里我们只是使用了TensorFlow Serving官方自带的一个测试模型，来告诉你怎么准备环境。在推荐模型实战的时候，我们还会基于TensorFlow构建多种不同的深度学习模型，到时候TensorFlow Serving就会派上关键的用场了。

那对于深度学习推荐系统来说，我们只要选择TensorFlow Serving的模型服务方法就万无一失了吗？当然不是，它也有需要优化的地方。在搭建它的过程会涉及模型更新，整个docker container集群的维护，而且TensorFlow Serving的线上性能也需要大量优化来提高，这些工程问题都是我们在实践过程中必须要解决

的。但是，它的易用性和对复杂模型的支持，还是让它成为上线TensorFlow模型的第一选择。

小结

业界主流的模型服务方法有4种，分别是预存推荐结果或Embedding结果、预训练Embedding+轻量级线上模型、利用PMML转换和部署模型以及TensorFlow Serving。

它们各有优缺点，为了方便你对比，我把它们的优缺点都列在了表格中，你可以看看。

模型服务方法	优点	缺点
预存推荐结果	简单、直接、快速	组合爆炸，无法引入线上场景特征
预训练Embedding	相比预存推荐结果降低了存储空间，线上部分实现简单、速度快	表达能力不强，无法引入线上场景特征
预训练Embedding+轻量级线上模型	兼具灵活性和线上推断效率，是主流的模型服务解决方案	不是end2end的解决方案，实现复杂度比较高
基于PMML的模型转换和上线方法	end2end的模型训练-服务解决方案	不能够完全支持所有复杂模型
TensorFlow Serving	end2end解决方案，支持绝大多数TensorFlow模型结构	只支持TensorFlow模型，线上服务效率较低，需要大量优化

我们的之后的课程会重点使用TensorFlow Serving，它是End2End的解决方案，使用起来非常方便、高效，而且它支持绝大多数TensorFlow的模型结构，对于深度学习推荐系统来说，是一个非常好的选择。但它只支持TensorFlow模型，而且针对线上服务的性能问题，需要进行大量的优化，这是我们在使用时需要重点注意的。

在实践部分，我们一步步搭建起了基于Docker的TensorFlow Serving服务，这为我们之后进行深度学习推荐模型的上线打好了基础。整个搭建过程非常简单，相信你跟着我的讲解就可以轻松完成。

课后思考

我们今天讲了如此多的模型服务方式，你能结合自己的经验，谈一谈你是如何在自己的项目中进行模型服务的吗？除了我们今天说的，你还用过哪些模型服务的方法？

欢迎在留言区分享你的经验，也欢迎你把这节课分享出去，我们下节课见！